Ingegneria del Software — SOLUZIONE DEL TEMA 26 Giugno 2020

Esercizio 1

Si consideri la seguente specifica di una classe mutabile `SequenzaVettori`, che rappresenta una sequenza (senza duplicazioni) di vettori bidimensionali. Un vettore è un elemento della classe `Vettore`, specificata più avanti.

```
public class SequenzaVettori {
  public SequenzaVettori(); //costruisce una sequenza vuota.

  //@ensures (* \result e' il numero di vettori nella sequenza *)
  public /*@ pure @*/ int size() {...}

  //@ensures (* \result e' il vettore i-esimo della sequenza se 0<=i<size() *).
  //@signals (IndexOutOfBoundException e) (i<0 || i>=size());
  public /*@ pure @*/ Vettore get(int i) throws IndexOutOfBoundException;

  //@ensures (* inserisce un nuovo elemento al termine della sequenza *)
  //@signals (DuplicateException e) (* eccezione se v esiste in this *)
  public void put(Vettore v) throws DuplicateException;

  //@ensures (* \result e' il vettore costituito dalla somma dei primi k vettori della sequenza *)
  //@signals (IndexOutOfBoundException e) (k<2 || k>size());
  public /*@ pure @*/ Vettore sum(int k);
}
```

La classe immutabile `Vettore` è così specificata:

```
public /*@ pure @*/ class Vettore {
  //costruisce un vettore dati modulo e fase
  public Vettore(double modulo, double fase);

  //@ restituisce il modulo del vettore
  public double modulo();

  //@ restituisce la fase del vettore
  public double fase();

  //@ restituisce il vettore somma di this e v
  public Vettore somma(Vettore v);
  ... altri metodi tipici ...
}
```

Domanda a)

Si scriva la specifica JML del metodo `put`.

Soluzione

```
//@ensures size() == \old(size())+1 &&
//@(\forall int i; 0<=i && i<\old(size());
//@    \old(!get(i).equals(v)) && get(i).equals(\old(get(i)))) &&
//@get(size()-1).equals(v);
//@
//@signals (DuplicateException e) size()==\old(size()) &&
//@(\exists int i; 0<=i && i<size(); \old(get(i)).equals(v)) &&
//@(\forall int j; 0<=j && j<\old(size()); get(j).equals(\old(get(j))));
public void put(Vettore v) throws DuplicateException;
```

Domanda b)

Si scriva la specifica JML del metodo `sum`.

Soluzione

```
//@ensures k>=2 && k<=size() &&
//@(k==2 ==> \result.equals(get(0).somma(get(1)))) &&
//@(k>2 ==> \result.equals(sum(k-1).somma(get(k))));
//@signals (IndexOutOfBoundException e) (k<2 || k>size());
public /*@ pure @*/ Vettore sum(int k);
```

Domanda c)

Si consideri una variante `SequenzaVettori2` della classe `SequenzaVettori`, che è identica alla classe precedente ma in cui la specifica di `put` è differente:

```
//@ensures (* inserisce il vettore v in una posizione qualunque *)
//@signals (DuplicateException e) (* eccezione se v esiste in this *);
public void put(Vettore v) throws DuplicateException e;
```

È possibile definire questa classe `SequenzaVettori` come derivata da `SequenzaVettori2`, rispettando il principio di sostituzione? Il viceversa? Motivare la risposta.

Soluzione

`SequenzaVettori` può essere sottoclasse di `SequenzaVettori2`, in quanto l'inserimento in coda alla sequenza è compatibile con l'inserimento in posizione qualunque (la postcondizione è più forte). Il viceversa invece viola la regola della postcondizione (la postcondizione è più debole).

Esercizio 2

Si consideri la seguente classe Java:

```
public class ConcTest {
  private double x, y;
  public ConcTest(double a) { x = y = a; }
  public synchronized void setX(double x) { this.x = x; }
  public synchronized double getX() { return x; }
  public synchronized double getY() { return y; }
  public synchronized void waitForEq() throws InterruptedException {
    while(x!=y) wait();
  }
}
```

Domanda a)

La sincronizzazione è corretta? In caso negativo cosa può succedere? E come si deve modificare il codice affinché la situazione critica non si verifichi?

Soluzione

Non possono esserci conflitti nell'accesso allo stato condiviso (gli attributi `x` e `y`), ma potrebbero verificarsi dei deadlock. Il thread chiamante il metodo `waitForEq` potrebbe infatti rimanere per sempre in stato di wait visto che nessun metodo invoca `notify` o `notifyAll`. La soluzione è inserire la chiamata al metodo `notifyAll` al termine del metodo `setX`.

Domanda b)

Si riscriva la classe `ConcTest` eliminando il metodo `waitForEq`, aggiungendo un metodo `setY` che permetta di variare il valore dell'attributo `y` e facendo in modo che due thread possano accedere (leggere o modificare) in parallelo uno l'attributo `x` e l'altro l'attributo `y`. L'implementazione deve comunque evitare conflitti tra diversi thread che accedono allo stesso attributo (`x` oppure `y`).

Soluzione

```
public class ConcTest {
  private double x, y;
  private Object lockX, lockY;
  public ConcTest(double a) {
    x = y = a;
    lockX = new Object();
    lockY = new Object();
  }
  public void setX(double x) {
    synchronized(lockX) { this.x = x; }
  }
  public double getX() {
    synchronized(lockX) { return x; }
  }
  public void setY(double y) {
    synchronized(lockY) { this.y = y; }
  }
  public double getY() {
    synchronized(lockY) { return y; }
  }
}
```

Esercizio 3

Si consideri il seguente programma Java.

```
public class Main {
  public static void main(String[] args) {
 1    Person p1 = new Student();
 2    Person p2 = new Person();
 3    Person p3 = new Teacher();
 4    Student p4 = new Student();
 5    Student p5 = new Teacher();
 6    Lecture lec1 = new Lecture();
 7    Lecture lec2 = new OnlineLecture();
 8    OnlineLecture lec3 = new OnlineLecture();

 9    lec1.addAttendant(p1);
10    lec1.addAttendant(p2);
11    lec1.addAttendant(p3);
12    lec1.addAttendant(p4);
13    lec1.addAttendant(p5);
14    lec2.addAttendant(p1);
15    lec2.addAttendant(p2);
16    lec2.addAttendant(p3);
17    lec2.addAttendant(p4);
18    lec2.addAttendant(p5);
19    lec3.addAttendant(p1);
20    lec3.addAttendant(p2);
21    lec3.addAttendant(p3);
22    lec3.addAttendant(p4);
23    lec3.addAttendant(p5);
  }
}

abstract class Person {
  public void join(Lecture lec) { System.out.println("Joining " + lec); }
  public void join(OnlineLecture lec) { System.out.println("Joining " + lec); }
}
class Student extends Person {
  public void join(Lecture lec) { System.out.println("Student joining " + lec); }
}
class Teacher extends Person {
  public void join(OnlineLecture lec) { System.out.println("Teacher joining " + lec); }
}
class Lecture {
  public void addAttendant(Person p) { p.join(this); }
  public String toString() { return "a lecture"; }
}
class OnlineLecture extends Lecture {
  public String toString() { return "an online lecture"; }
}
```

Domanda a)

Quali linee del programma non sono valide? Motivare brevemente la risposta.

Soluzione

Linea 2 in quanto `Person` è una classe astratte e linea 5 in quanto `Teacher` non è sottoclasse di `Student`.

Di conseguenza vanno eliminate tutte le righe che contengono `p2` o `p5`, ovvero le linee 10, 13, 15, 18, 20, 23.

Domanda b)

Dopo aver rimosso tutte le linee non valide identificate al punto precedente, indicare cosa stampa il programma, motivando brevemente la risposta.

Soluzione

(Linea 09) Student joining a lecture
(Linea 11) Joining a lecture
(Linea 12) Student joining a lecture
(Linea 14) Student joining an online lecture
(Linea 16) Joining an online lecture
(Linea 17) Student joining an online lecture
(Linea 19) Student joining an online lecture
(Linea 21) Joining an online lecture
(Linea 22) Student joining an online lecture